

05 · 多模型 Provider 如何统一抽象

从 Model/Context/Message 到统一流协议，再到 OpenAI、Anthropic、Google 适配器

分析版本: 027a5847901b5dde30270abaa1041046cd2b4b55

核心结论

pi-ai 的统一抽象有三层：数据协议（Model/Message/Context/Tool）、调用协议（Provider.stream/streamSimple 与 StreamOptions）、事件协议（AssistantMessageEventStream）。具体 Provider adapter 在边界两侧做双向转换：统一 Context -> 供应商 payload；供应商原生流事件 -> 统一 AssistantMessageEvent。

1. 为什么 Provider 与 API 是两个概念

types.ts 同时定义 ProviderId 和 Api。Provider 表示服务来源（openai、anthropic、google、openrouter 等）；Api 表示协议形状（openai-responses、anthropic-messages、google-generative-ai 等）。同一个 Provider 可能使用不同 API，同一个 API 形状也可被兼容服务复用。

Model 因而同时携带 provider、api、baseUrl、id、input capability、reasoning、contextWindow、maxTokens、cost、compat 等元数据。调用层按 provider 找 Provider，再按 model.api 走相应 stream 实现。

2. 统一消息模型

统一类型	核心字段	为什么存在
UserMessage	role=user, text/image content, timestamp	统一文本与多模态输入
AssistantMessage	text/thinking/toolCall blocks + provider/model/usage/stopReason	把不同供应商输出压成同一结果
ToolResultMessage	toolCallId/toolName/content/isError	下一轮把本地工具执行结果反馈给模型
Context	systemPrompt/messages/tools	Provider adapter 的统一输入
Tool	name/description/parameters	不同工具 schema 的最小公共结构

3. 统一事件协议是流式兼容的关键

AssistantMessageEvent 把原生 SSE/SDK event 归一为 start、text_start/delta/end、thinking_start/delta/end、toolcall_start/delta/end、done/error。上层 Agent Loop 因而只写一套 switch，不需要知道 OpenAI 是 response.output_text.delta，Anthropic 是 content_block_delta，Google 又是另一种 chunk。

4. StreamFunction 的错误契约

pi-ai 的设计要求请求/模型/runtime failure 在 stream 调用后尽量通过事件协议编码，而不是让调用方依赖不同 SDK 的异常形状。终态 error 需要携带 stopReason=error/aborted 与 errorMessage。Agent Core 的 StreamFn 注释重复了这个约束，说明这是跨包重要契约。

5. stream 与 streamSimple 为什么同时存在

stream 暴露 API-specific options；streamSimple 提供跨 Provider 的“简单选项”，例如 reasoning level、thinking budgets、temperature、maxTokens。具体 adapter 将统一 reasoning 映射为自身参数，例如 OpenAI reasoning effort、Anthropic adaptive/budget thinking。

6. OpenAI Responses：统一 Context -> ResponseCreateParams

1. createClient 使用 model.baseUrl、apiKey、headers 和 session affinity 创建 OpenAI SDK client。
2. convertResponsesMessages 将统一 messages 转成 Responses input；systemPrompt 转为 system/developer。
3. convertResponsesTools 将统一 Tool/schema 转成 function/custom tools。
4. buildParams 添加 model、input、stream=true、cache、reasoning、max output token 等。
5. onPayload 可以在发送前检查/替换 payload。
6. client.responses.create 发出请求；processResponsesStream 把原生事件转回统一事件。

7. Anthropic adapter 的不同点如何被压在边界内

Anthropic adapter 自己处理 base64 image content、cache_control、thinking/adaptive thinking、tool casing/兼容性、beta headers、Anthropic SDK RawMessageStreamEvent 等。这些差异不会泄漏到 Agent Loop：上层仍然只看到 Context 和统一 AssistantMessageEvent。

这说明 compat 字段不是“杂项”，而是 Provider abstraction 的现实组成部分：兼容服务往往协议大体相同，但对 developer role、strict tool、cache、thinking、session affinity 等细节不同。compat 将这些差异显式数据化，减少 scattered if(provider===...)。

8. Provider 注册与 ModelRuntime

Coding Agent 的 ModelRuntime 包装 pi-ai Models：内置 provider catalog、models.json 覆盖、Extension Provider、远程 catalog 和 credential resolution 最终组合为 Provider。请求前 prepareRequest 做 provider lookup、认证解析、baseUrl override、headers/env 合并，再调用 provider.streamSimple。

9. 上层为什么能切模型而不改 Agent Loop

Agent Loop depends on:

```
Model<Api>
Context
StreamFn
```

Provider-specific logic lives behind StreamFn.

Changing openai -> anthropic changes model/provider adapter, not runLoop.

10. 新增 Provider 的修改面

- 定义/复用 API adapter: 负责 payload 与 stream event 的双向转换。
- 提供 Provider 对象与模型 catalog/认证逻辑。
- 只有真正不同的能力用 compat/model metadata 表达, 不要让 coding-agent 到处判断 provider 名。
- 补充跨 provider 的消息、工具、thinking、usage、error、cache 测试。
- 验证 foreign model/history replay, 尤其 tool-call id、thinking signature 和角色兼容。

11. 容易出现 Bug 的边界

- 工具调用 ID 在跨 Provider/跨模型历史重放时格式不兼容。
- stream 原生事件漏掉 terminal event, 导致 stopReason 长期 pending。
- usage 对 cached/reasoning token 的统计口径不同。
- Provider 声称兼容 OpenAI, 但不支持 developer/strict/store/reasoning 字段。
- thinking signature/加密内容未保留, 下一轮 API 无法继续推理上下文。
- headers/baseUrl/auth overlay 合并顺序错误导致 credential 或代理失效。

动态验证方案

实验 1：验证统一事件

假设：不同 Provider 对同样文本都映射成 text_start/delta/end + done

操作：用 mock 或真实低成本模型分别调用 OpenAI/Anthropic, 打印 AssistantMessageEvent.type。

预期：上层事件类型集合一致, 只有 provider/model/usage 等内容差异。

若不一致：若某 adapter 缺 start/done, Agent UI 生命周期可能异常。

实验 2：验证 payload 边界

假设：Agent Loop 不出现 Provider 专用请求字段

操作：在 before_provider_request 打印 payload, 同时搜索 packages/agent 是否含 Responses/Anthropic SDK 类型。

预期：Provider 字段只出现在 pi-ai adapter; agent-core 只见统一 Context。

若不一致：若 agent-core import SDK, 说明抽象边界被破坏。

源码证据索引

文件	关键符号/用途	本报告结论
packages/ai/src/types.ts	Api/ProviderId/Message/Context/ AssistantMessageEvent/StreamOptions	统一协议定义
packages/ai/src/models.ts	Models / model registry	模型发现与 Provider 绑定
packages/ai/src/api/openai-responses.ts	streamSimple / buildParams / SDK create	OpenAI 适配示例
packages/ai/src/api/openai-responses-shared.ts	convertResponsesMessages / processResponsesStream	双向协议转换
packages/ai/src/api/anthropic-messages.ts	Anthropic content/thinking/cache/tool adapter	证明差异被封装在 adapter
packages/coding-agent/src/core/model-runtime.ts	prepareRequest / streamSimple	产品层的 Provider 组合与认证

证据状态：除非单独标注，本报告中的行为结论均来自上述固定 commit 的静态源码与仓库测试/文档交叉验证；未实际启动 Pi、未抓取真实网络请求，因此“运行结果已验证”项为空。

推荐阅读顺序

优先级	文件	阅读目的	精读
1	packages/ai/src/types.ts	先背熟统一协议	是
2	packages/ai/src/models.ts	理解 Models/Provider registry	是
3	packages/ai/src/api/openai-responses.ts	选一个 Provider 精读请求构造	是
4	packages/ai/src/api/openai-responses-shared.ts	精读消息/事件转换	是
5	packages/ai/src/api/anthropic-messages.ts	对比第二种协议，找差异点	否